

Universidad de San Andrés, *Buenos Aires, Argentina*



I206 - Inferencia y Estimación

Trabajo Práctico II

Teoría de la Información:

Codificación de Huffman

Camiscia, Luca

lcamiscia@udesa.edu.ar

Palavecino, Axel

apalavecino@udesa.edu.ar

Pereyra, Agustín

apereyra@udesa.edu.ar

21 de octubre de 2025

Abstract

En este presente trabajo se aborda experimentalmente este principio, investigando cómo se desempeña en la práctica el algoritmo de **Codificación de Huffman**, donde se investiga cómo la capacidad predictiva de una fuente determina su potencial compresión, esto se hace mediante el análisis de textos con propiedades de ocurrencia de caracteres radicalmente diferentes (lenguaje natural, datos estructurados y secuencias genómicas), además de demostrarse empíricamente la optimalidad de este método. Los resultados cuantifican la eficiencia del algoritmo, que minimiza la longitud de codificación promedio ... (**Aca van el resto de los resultados**)

1. Introducción

En la era del **Big Data**, obtener capacidad de representar información de manera concisa y compacta es un desafío fundamental. La *Teoría de la Información*, la cual comenzó **Claude Shannon**,

postula que la compresibilidad de cualquier dato está intrínsecamente limitada por su contenido de incertidumbre, esta medida es cuantificable, y es conocida como *entropía*.

2. Desarrollo Experimental

2.1. Estimación del Modelo probabilístico de la Fuente

El primer paso en la implementación de la **codificación de Huffman** es estimar el modelo probabilístico de la fuente (texto). Esto se logra mediante el análisis de la frecuencia de aparición de cada símbolo en el conjunto de datos. La probabilidad estimada p_i del i -ésimo símbolo s_i se calcula como:

$$p_i = \frac{f_i}{\sum_{j=1}^N f_j} \quad (1)$$

donde cada caracter es una realización de la variable aleatoria fuente S . El objetivo en esta etapa es estimar una **función de masa de probabilidad (PMF)** de cada fuente de texto. El resultado de este paso es un conjunto de pares (s_i, p_i) que representan los símbolos y sus probabilidades de ocurrencia, es decir, su **probabilidad empírica** de aparición $p(x)$. Este diccionario constituye el modelo estadístico de la fuente, y es el insumo fundamental sobre el cual operará el algoritmo de **Huffman**.

Es de interés aclarar que en este informe no se realiza un distinción entre mayúsculas y minúsculas, ya que el objetivo principal es reducir el tamaño del texto manteniendo su compresibilidad. La información que aporta una diferenciación entre mayúscula y minúscula es muy pequeña al lado de la complejidad que le aporta a la codificación de Hoffman. La ausencia de mayúsculas al inicio de las oraciones no afecta la comprensión, por lo que no se considera prescindible. No obstante, posteriormente podría aplicarse un algoritmo que reemplace automáticamente por mayúsculas las letras que sigan a un punto (“.”).

Las características del resultado de la Ecuación (1) se basan en los **Axiomas de la Probabilidad**,

es decir, la suma de los p_i es igual a 1, $\sum_{i=1}^N p_i = 1$, la probabilidad de un evento puntual es mayor a cero, $\mathbb{P}(p_i) \geq 0$, y si $A \cap B = \emptyset \Rightarrow \mathbb{P}(A \cup B) = \mathbb{P}(A) + \mathbb{P}(B)$

2.2. Codificación de Huffman

La característica que diferencia este código de otros universales como el *código Morse* es que ninguna cadena de código asignada a algún símbolo es prefijo de otra. A esta propiedad se la conoce como *prefix-free* y es gracias a esto que su decodificación es inmediata.

El código se construye como un árbol binario, cuyas hojas representan un símbolo y a cada rama se le asocia a un elemento binario (generalmente a la rama izquierda el 0 y a la derecha el 1) y el algoritmo para construirlo es muy sencillo:

Algorithm 1 HuffmanCoding

Input: PMF de los símbolos s_i de la cadena S

Output: Codificación Hoffman para S

```
1:  $Q \leftarrow$  lista de tuplas ( $symbol, p_i | subtree$ )
2: while  $Q.size() > 1$  do
3:   Ordenar  $Q$  por  $p_i$ 
4:    $a, b \leftarrow$  los 2 símbolos de  $< p_i$ 
5:   Agregar 0 y 1 a los prefijos de  $a$  y  $b$  respectivamente
6:   Fusionar los subárboles
7:   create node  $c$  with children  $a$  and  $b$  and weight  $a.weight + b.weight$ 
8:    $Q.push(c)$ 
9: end while
10: return code obtained from the final tree
```

2.3. Construcción del Árbol de Huffman Reducción de Memoria

2.4. Medidas de Desempeño y Análisis Cuantitativo

$$R = \left(1 - \frac{L}{L_{\text{unif}}}\right) \times 100 \% \quad (4)$$

Longitud Promedio de Codificación (L):

$$L = \sum_{i=1}^N p_i l_i \quad (2) \quad \text{Compresibilidad de la Fuente (Entropía Normalizada, } \eta)$$

Comparación con la Codificación Uniforme

$$L_{\text{unif}} = \lceil \log_2(N) \rceil \quad (3) \quad \eta = \frac{H(S)}{\log_2(N)} \quad (5)$$

3. Análisis de Datos

4. Conclusión